

TEKNOFEST
HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ
ULAŞIMDA YAPAY ZEKA YARIŞMASI
KRİTİK TASARIM RAPORU

DEEP-AI

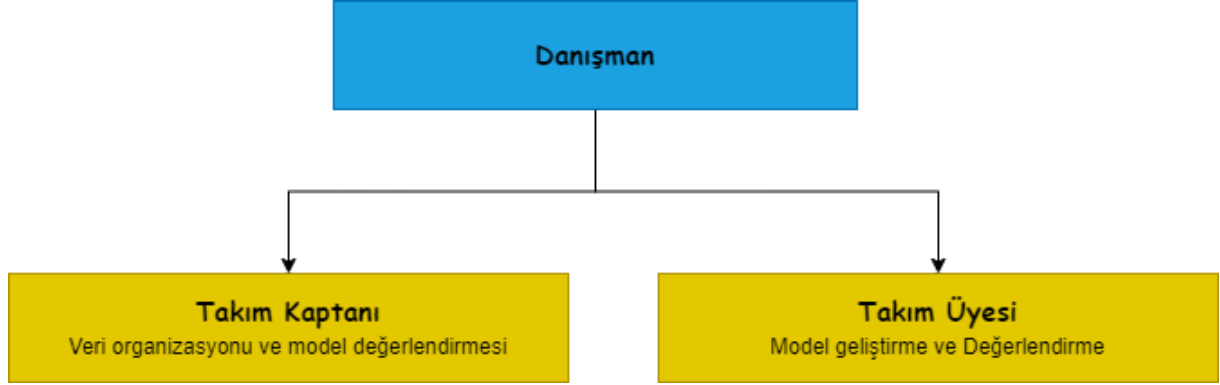
404110

İçindekiler

1. Takım şeması	3
2. Proje Mevcut Durum Değerlendirmesi	3
3. Algoritmalar ve Sistem Mimarisi	4
3.1 Veri Setleri	4
3.2 Algoritmalar	4
4. Özgünlük	8
5. Sonuçlar ve İnceleme	9
6. Kaynakça	11

1. Takım şeması

DEEP-AI takımına ait takım üyeleri ve görevlerini içeren takım şeması Şekil 1’de verilmiştir. DEEP-AI takımı 1 kaptan, 1 takım üyesi ve 1 danışmandan oluşmaktadır.



Şekil 1. Takım şeması

Gösterildiği üzere danışman hariç, verilerin organizasyonu, model geliştirme ve değerlendirme olmak üzere 2 adet görevlendirmeden oluşmaktadır.

Verilerin organizasyonu; Veri seti oluşturulması için görüntülerin bulunması, aykırı görüntülerin çıkarılması, etiketlenmesi ve model eğitimi için uygun formata getirilmesidir.

Model geliştirme ve Değerlendirme; Problemin çözümünde kullanılacak modellerin belirlenmesi, veri setinin kullanılarak eğitiminin yapılması ve sonuçların belirlenen metriklere göre değerlendirilmesidir.

Ekip üyeleri kendi görev dağılımlarının yanı sıra daima birbirleriyle koordine ve yardımlaşarak bu süreci yürütmektedirler.

2. Proje Mevcut Durum Değerlendirmesi

DEEP-AI takımı ÖTR (ön tasarım raporu) yazarken proje tasarımını olabildiğince mümkün tercihler yapmıştı. Bu nedenle ÖTR aşamasında hazırlanan proje takvimi uyarınca hareket edildiğinden büyük bir sorun yaşanmadan hedeflere ulaşılmıştır.

Yarışma komitesi tarafından verilen video da dahil olmak üzere çeşitli ortamlardan (özellikle Youtube, Dailymotion, Vimeo, vs) drone ile çekilmiş videolar indirilmiş ve bu videolardan görüntüler elde edilmiştir. Daha sonra bu görüntülerden amaca uygun olanlar ayıklanmıştır. Elde edilen görüntülere veri artırma işlemleri (data augmentation) python kütüphanesi olan albumentations kullanılarak uygulanmıştır. Görüntüler etiketlenirken her türlü etiketleme yapısına (pascal-voc, yolo) olanak sağlayan LabelImg [1] kullanılarak, araba, kamyon, motosiklet, insan, otobüs sınıfları ile etiketlenmiştir.

ÖTR aşamasında belirtildiği üzere araba, insan, kamyon, otobüs vb. nesneleri tespit edebilen nesne tespit modeli ve iniş yerlerini tespit edebilen iniş yeri tespit modeli olmak üzere 2 modelli yaklaşım uygulanmıştır.

Eğitim ve yarışmada kullanılacak donanımlarda da ÖTR’ye göre değişikliğe gidilmemiştir. Eğitim yine Google Colab servisi ile bulut üzerinden, yapılacaktır. Test ve yarışmada kullanılacak donanım ise NVIDIA GEFORCE RTX 3050 ekran kartı, AMD RYZEN 5 5600h işlemci, 16 GB RAM ve Windows 10 işletim sistemi bulunduran HP dizüstü bilgisayardır. Google Colab ortamında bazen yaşanan bazı kullanım kısıtlamalarından dolayı alternatif olarak Kaggle ortamı da eğitim işlemlerinde kullanılmıştır.

Model test aşamasında her bir model yaklaşık 1000 adet görüntüde kendi bilgisayarlarında geçen zaman ve doğruluk metrikleriyle test yapmışlardır.

ÖTR raporunda belirtildiği üzere SAHI (Slicing Aided Hyper Inference) [2] algoritması da doğruluğu arttırmak amacıyla entegre edilip sonuçlar incelenmiştir. Fakat sonuçlar incelendiğinde hız faktörü göz önünde bulundurulduğunda kullanma konusunda tereddüt içinde kalmıştır.

3. Algoritmalar ve Sistem Mimarisi

3.1 Veri Setleri

Yarışma veri seti: Yarışma komitesi tarafından İHA (insansız hava aracı) ile çekilmiş videolar verilmiş ve bu videolar görüntülere çevrilmiştir. Yakalılık 800 adet olan bu görüntüler içerdikleri nesnelere göre etiketleme yapıldıktan sonra yapay zeka modellerinin eğitiminde kullanılmıştır. Yarışma tarafından verilen bu veri setinin ne tür verilere ihtiyaç olduğu konusunda ve test verilerinin de bu yapıda geleceği sebebiyle kullanılmıştır.

İnternetteki kuşbakışı İHA videoları: İnternette veri sayısını ve çeşitliliğini arttırmak için yapılan araştırmalarda İHA ile kuşbakışı çekilmiş, içerisinde insan, kamyon, araba vb. nesneleri içeren 3 adet video bulunmuştur. Bu videolardan görüntüler elde edilmiştir. Bu görüntülerden eğitime uygun olmayacak görüntüler ayıklanmış, veri tekrarını önlemek amacıyla düzenlenip, etiketlenip model eğitiminde kullanılmıştır..

VisDrone veri seti: VisDrone veri seti İHA ile kuşbakışı çekim yapılan, içerisinde gece gündüz, yağmurlu güneşli sisli havada çekilmiş, kişi, insan, araba, otobüs, kamyon, motor ve bisiklet gibi nesneleri barındıran bir yarışma verisidir. Bu veri setinin içerisinde kullanıma hazır görüntüler bulunmaktadır [3]. Aykırı görüntüler ayıklandıktan sonra model eğitiminde kullanılmıştır.

Veri augmetasyonu ve Fotoşop: Yarışma komitesi tarafından verilen videoda UAP ve UAİ alanlarının yetersiz olduğu gözlemlenmiş ve bir python kütüphanesi olan albumentations kullanılarak veri çoğaltma yoluna gidilmiştir. Fakat UAP ve UAİ alanlarının bulunduğu ortamların çeşitlilik bakımından az olması sebebiyle fotoşop yöntemleri ile yapay resimler oluşturulmuştur. Veri çoğaltma ve fotoşop işlemlerinden sonra etiketlenen görüntüler model eğitiminde kullanılmıştır.

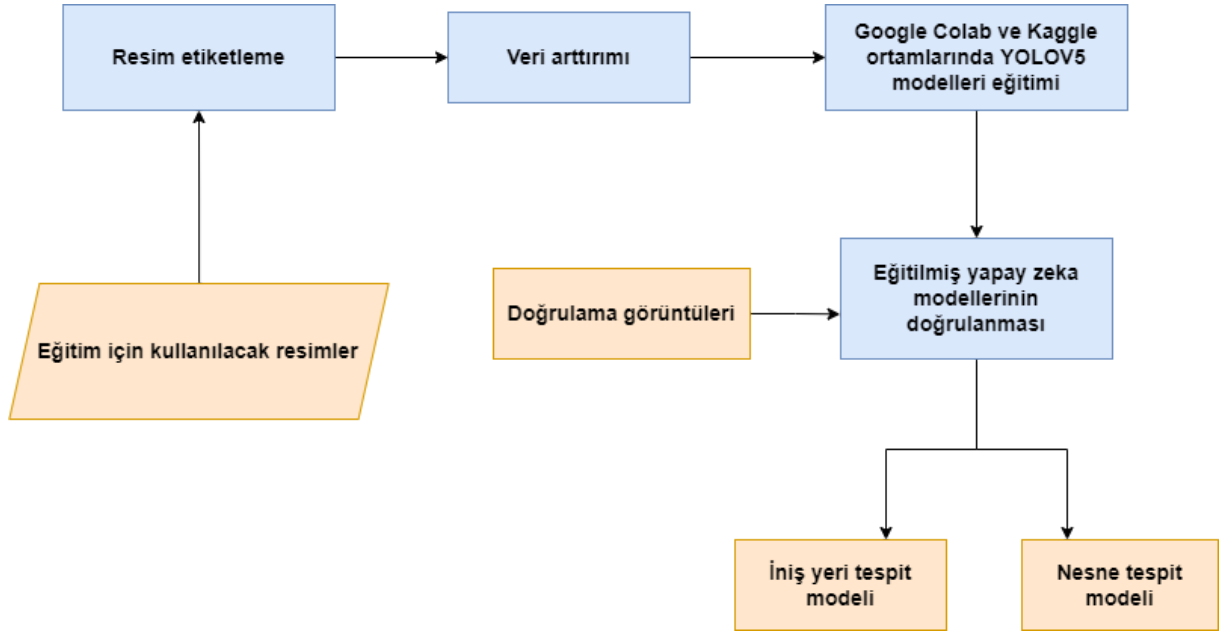
Parking Lot Database: Bu veri seti 1280x720 boyutunda farklı açılarda ve hava koşullarında çekilmiş görüntüler içermektedir [4]. Bu veri seti kamera açısı ve nesne boyutları bakımından yetersiz görülmüştür. Aynı zamanda veri tekrarı ve etiketleme bakımından çok yetersiz bulunmaktadır. Fakat içerisinden uygun olan görüntüler alınıp elle etiketleme yapılmıştır.

3.2 Algoritmalar

Yarışma probleminin çözümü için birbirine karışması muhtemelen nesneleri engellemek ve modellerin kendilerine verilmiş spesifik görevlere odaklanması amacıyla 2 modelli yaklaşım benimsenmiştir. Birinci model araba, kamyon, insan, motorsiklet vb. nesneleri tespit edebilen nesne tespit modelidir. Bu nesne tespit modelinin verdiği koordinatlar herhangi bir uçan arabanın iniş yapamayacağı alanlardır.

İkinci model olarak UAP ve UAİ alanlarını tespit edebilen iniş yeri tespit modelidir. İniş yeri tespit modeli araçların iniş yapabileceği alanlardır. Eğer iniş yeri olarak tespit edilen alanlarda nesne tespit modelinin çıktılarında herhangi biri var ise iniş yapılamaz olarak kontrol edilmektedir.

Yapay zeka modelleri geliştirilirken Şekil 2'de verilen akış şeması takip edilmiştir.



Şekil 2. Akış şeması

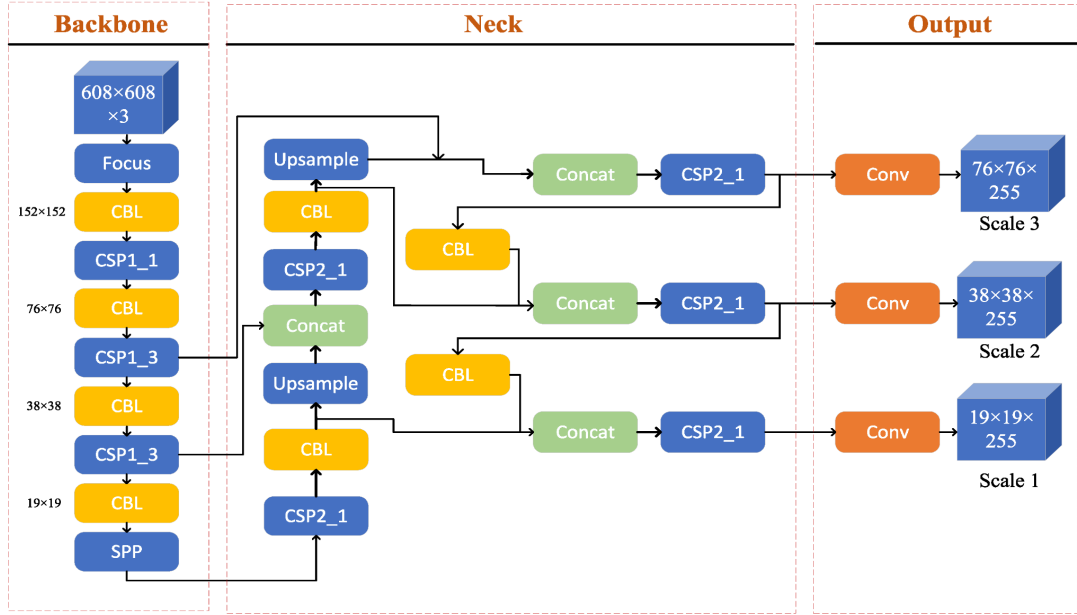
Eğitim işleminin gerçekleşebilmesi için uygun olan olan görüntüler bir araya getirilmiştir. Bir araya getirilen görüntüler LabelImg programı kullanılarak etiketlenmiştir. Görüntülerin sayısının yetersiz olması sebebiyle veri artırma (Veri augmentasyon) yöntemleri ile sayıları artırılmıştır. Veriler hazır hale getirdikten sonra Google Colab ve Kaggle ortamlarında ücretsiz GPU sağlamaları sebebiyle eğitim işlemi gerçekleştirilmiştir. Eğitilen modellerin performansı için bir doğrulama veri seti oluşturulmuştur. Bu veri seti ile değerlendirme metriklerinin sonuçları incelenmiştir. Modeller değerlendirildikten sonra İniş yeri tespit ve Nesne tespit modeli olmak üzere 2 adet model meydana gelmiştir..

Eğitilen modellerin yarışma esnasında çalışma kurguları;

- Görüntünün modellere uygun boyuta getirilmesi
- Tahminleme için hem nesne tespit hem de iniş modellerine girdi olarak verilmesi
- Tahminlenen bölgelerin ve sınıfların elde edilmesi
- Nesne tespit çıktılarının tümü iniş yapılamaz olması
- İniş yerinin tespit ettiği bölgelerde nesne tespit çıktılarının kontrol edilmesi
- İniş yerinin uygunluğunun belirlenmesi
- Tüm sonuçların bir json dosyasında bir araya getirilmesi

İniş yeri tespit modeli ve nesne tespit oluşturulan veri setleri ile eğitme işlemi YOLOv5 mimarisi ile transfer öğrenme yaklaşımı benimsenerek yapılmıştır. Transfer öğrenme yaklaşımı sayesinde saatlerce süren bir eğitim yapmak yerine bazı katmanlar dondurulup eğitim işlemine dahil etmeyerek daha kısa sürede eğitim gerçekleşmiş olur. Literatür incelendiğinde doğruluğu ve başarısı kanıtlanmış YOLOv5, SSD (Single Shot Detector), Detectron 2, Faster-RCNN olmak üzere çeşitli nesne tespit modelleri bulunmaktadır. Bu nesne tespit modellerinin her biri kendi mimarisinin sağladığı hız veya doğruluk gibi çeşitli avantaj veya dezavantajları bulunmaktadır. Literatür incelendiğinde YOLOv5 mimarisinin hız ve doğruluk açısından öne çıktığı anlaşılmaktadır [5]. Aynı zamanda ön tasarım raporunda sunulan ön çalışmada YOLOv5 mimarisinin az bir veri olmasına rağmen etkili sonuçlar ve tek görüntüde 0.3 saniye gibi kısa bir tahminleme süresine sahip olması tercih sebeplerinden olmuştur.

YOLOv5 mimarisi Şekil 3'de verildiği gibi Backbone, Neck ve output olmak üzere 3 ana kısımdan oluşmaktadır.



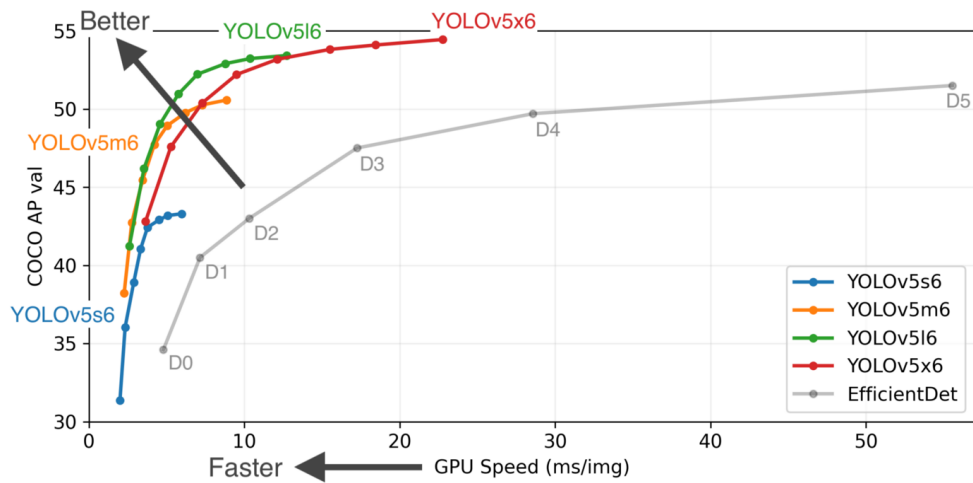
Şekil 3. YOLOV5 mimarisi

Backbone ağı cross-stage partial network (CSP) spatial pyramid pooling (SPP) katmanlarında çoklu konvolüsyon ve ortaklama (pooling) işlemlerini kullanarak farklı boyutlardaki girdi görüntülerinde özellik haritasını(feature map) çıkarmaktadır. SPP yapısı aynı özellik haritası için farklı ölçeklerde özellik çıkarımı gerçekleştirirken, BottleneckCSP hesaplama miktarını azaltıp, tahminleme hızını artırır ve sonuç olarak tespit işlemi için doğruluğu arttıran 3 adet özellik haritası oluşur [6][7].

Neck ağında FPN (feature pyramid network) ve PAN ağları kullanılır. FPN güçlü özellikleri üst özellik haritelerinden alt özellik haritalarına iletir. Aynı zamanda PAN yapısı güçlü yer özelliklerini alt özellik haritelerinden üst özellik haritalarına iletir. Bu iki yapı Backbone ağından gelen çıkarılmış özellikleri birlikte güçlendirir ve tespit etme kabiliyetini arttırlar [6][7].

Output yapısında farklı boyutlardaki özellik haritalarının tahminlemesi yapılır.

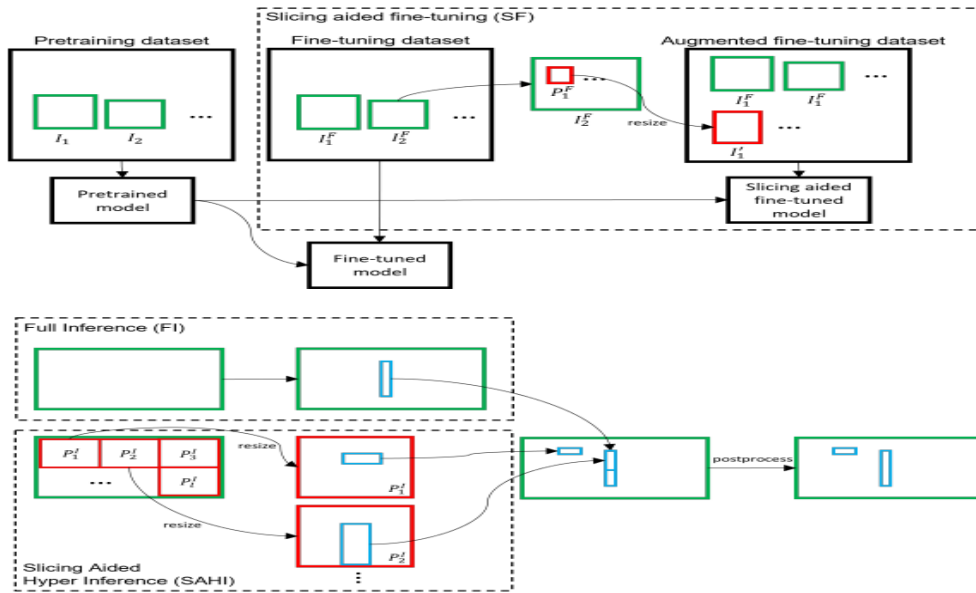
YOLOv5, YOLOv5s, YOLOv5m, YOLOv5l, and YOLOv5x olmak üzere 4 adet mimariden oluşur. Bu yapıların aralarındaki temel fark özellik çıkarıcı modüllerin ve konvolüsyon çekirdeklerinin sayısıdır.



Şekil 4. YOLOv5 mimari karşılaştırılması

Özellik çıkarıcı modüllerin ve konvolüsyon çekirdeklerinin sayısı YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x olmak üzere artış göstermektedir. YOLOv5 mimarilerinin hız ve doğruluk karşılaştırması Şekil 4.'de verilmiştir. Grafik incelendiğinde YOLOv5 mimarilerinde konvolüsyon ve özellik çıkarıcı modüllerin sayısı arttıkça doğruluğun arttığı fakat hız performans açısından bir dezavantaj getirdiği anlaşılmaktadır. Grafik incelendiğinde, hız performans ve doğruluk faktörü göz önünde bulundurulduğunda optimum model olarak YOLOv5m mimarisinin olduğu belirlenmiştir ve hem iniş yeri tespit modeli, hem de nesne tespit model mimarisi olarak YOLOv5m kullanılması uygun görülmüştür.

Bilgisayarla Görme alanındaki açık ara en önemli uygulama alanlarıdır. Bununla birlikte, küçük nesnelerin tespiti ve büyük görüntüler üzerinde çıkarım, pratik kullanımda hala önemli sorunlardır. Bu sebeple altınuç ve arkadaşlarının geliştirdiği SAHI algoritması yapay zeka modellerimize entegre edilip test edilmiştir. SAHI algoritması dilimlenmiş çıkarım kavramı (Saliance Aware) temel olarak Şekil 5.'de verildiği gibi orijinal görüntünün daha küçük parçaları üzerinde çıkarım yapmak ve ardından dilimlenmiş tahminleri orijinal görüntü üzerinde birleştirmektedir. SAHI algoritması YOLO, Detectron gibi modellerle entegre çalışabilmektedir [2].



Şekil 5. SAHI algoritması

4. Özgünlük

- Bu çalışmayı gerçekleştirirken yarışma komitesi tarafından verilen verilerin haricinde internet üzerinden bulunan görüntü verileri takım üyeleri tarafından etiketlenip model eğitimi amacıyla kullanılmıştır.
- İniş yeri tespitinde kullanılacak verilerin sayısı halen yetersiz olduğundan Şekil 6'de verilen görüntüdeki gibi fotoğraf ile yapay görüntüler oluşturulmuştur.



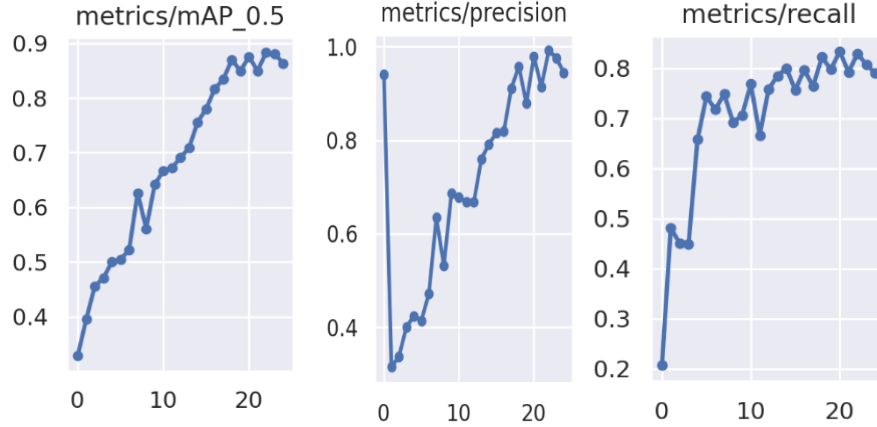
Şekil 6. Fotoğraf ile yapay görüntü

- Literatür incelendiğinde veri sayısı ve çeşitliliğinin kısıtlı olduğu durumlarda döndürme, parlaklık, yakınlaştırma ve uzaklaştırma gibi çeşitli data augmentation yöntemlerinin uygulandığı sistemlerde model başarılarının arttığı gözlemlenmiştir [8]. Bu sonuçlara göre, bu çalışmada verilere bulunan görüntülere ek olarak data augmentation yöntemlerini uygulayarak ayrıca veri sayısı ve çeşitliliğinin artırılması sağlanmıştır.
- Bu çalışmada özgün olarak yapay zekâ yöntemini geliştirirken 2 modelli bir yaklaşım belirlenmiştir. Birinci model görüntülerde bulunan taşıt ve insan nesnelerini tespit (koordinat ve sınıfları) etmeye odaklanırken, ikinci model UAP, UAI alanlarını tespit etmeye odaklanacaktır.
- Modellerin eğitimi aşamasında başarının artırılması ve eğitim süresinin kısaltılması amacıyla modellerde bulunan belirli katmanların dondurulmasıyla transfer öğrenme yaklaşımı uygulanmıştır.
- Yapay zeka modelleri genel olarak başarılı sonuçlar verse de küçük boyutlu nesneleri içeren bazı durumlarda tespit işleminde başarısız olmaktadır. Küçük boyutlu nesne başarı probleminin çözümü için SAHI algoritması yapay zeka modellerine entegre edilip test edilmiştir.

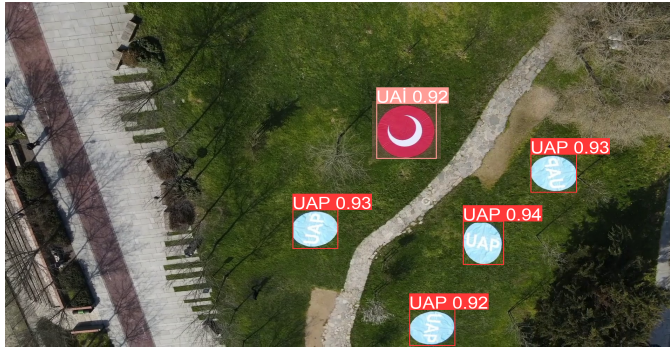
5. Sonular ve İnceleme

Yapay zeka modelleri eęitilirken ve test edilirken TR raporunda belirtildięi gibi recall, precision, mAP metrikleri dikkate alınmıřtır.

İniř yeri tespit modelinin eęitiminde mAP deęeri 0.86, precision 0.87, recall deęeri ise 0.79 olarak řekil 7’de grsel bir řekilde gsterilmiřtir.



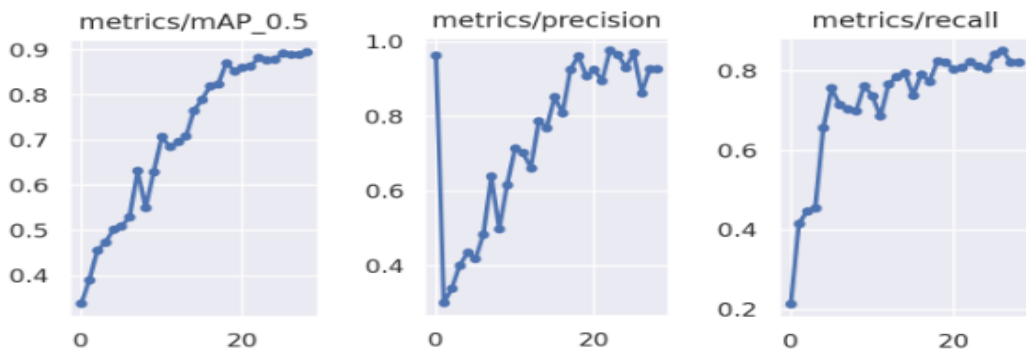
řekil 7. İniř yeri tespit model eęitim sonuları



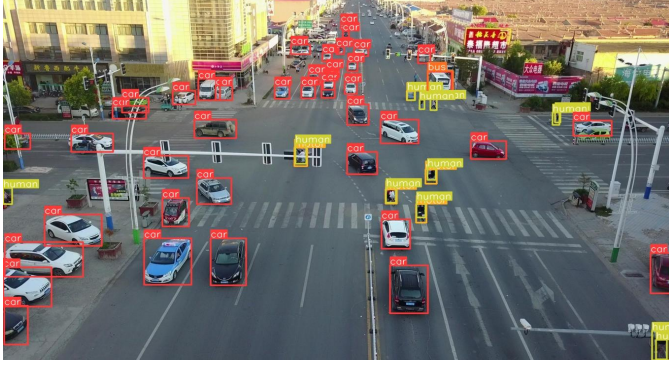
řekil 8. İniř yeri rnek sonu

- İniř yeri tespit modelinin tespit ettięi alanlar ve doęruluk deęerleri verilmiřtir.

Nesne tespit modeli eęitilirken řekil 9’de gsterildięi gibi elde edilen mAP deęeri yaklaşık olarak 0.90, precision 0.85, recall deęeri ise 0.81 olarak elde edilmiřtir.



řekil 9. Nesne tespit model eęitim sonular



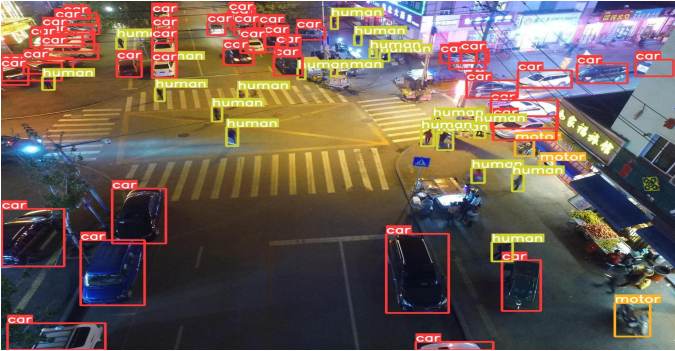
Şekil 10.. Nesne tespit modeli örnek sonuç

- Araçların ve insanların bulunduğu nesne tespit modelinin sonuçları Şekil 10. 'da verilmiştir.



Şekil 11. Nesne tespit modeli örnek sonuç

- İçinde insanların bulunduğu nesne tespit modelinin örnek çıktıları Şekil 11.'de verilmiştir.



Şekil 12. Nesne tespit model örnek sonuç

- Karanlık koşullarda nesne tespit modelinin test görüntüsündeki sonuçları Şekil 12'de verilmiştir.

inference_time: 237.78

1760 adet görüntüde hem nesne tespit modelinin hem de iniş yeri tespit modelinin birlikte çalıştığı, SAHI algoritmasının olmadığı test sisteminde toplam geçen zaman 237.78 saniyedir (3.96 dakika). Her bir görüntünün tahminleme süresi ortalama 0.13 saniyedir. Bu süre gayet ideal bir süredir. Fakat takım bu süreyi daha hızlı hale getirmek için kritik tasarım raporunu teslim ettikten sonra Python yerine C++ programlama dilini kullanarak tahminleme yapmayı deneyeceklerdir. Çünkü C++ dilinin Python diline göre 3 kat daha hızlı olduğu bilinmektedir.

Modeller/Metrics	Görüntü Sayısı	Tahminleme Süresi (Dakika)	mAP	Recall	Precision
İniş yeri tespit	1760	2.30	0.976	0.985	0.984
Nesne tespit	1065	2.08	0.888	0.815	0.915

Tablo 1. Test sonuçlar



Şekil 13. SAHI olmadan nesne tespit

- SAHI algoritması olmadan Şekil 13'te verildiği küçük boyutlardaki test görüntülerinde bazı nesneler tespit edilememektedir.



Şekil 14. SAHI entegreli nesne tespit sistemi

- SAHI algoritması ve nesne tespit modelinin entegre edilip çalıştırıldığı test Şekil 14'de verilen görüntüsünde Şekil 13'de tespit edilen çıktılara göre daha yüksek doğruluk değerleri ve tespit edilemeyen nesneleri tespit edebildiği gözlemlenmiştir.

6. KAYNAKÇA

[1] <https://github.com/tzutalin/labelImg>

[2] Akyon, F. C., Altinuc, S. O., & Temizel, A. (2022). Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection. arXiv preprint arXiv:2202.06934.

[3] Zhu, P., Wen, L., Du, D., Bian, X., Ling, H., Hu, Q., ... & Song, Z. (2018). Visdrone-det2018: The vision meets drone object detection in image challenge results. In Proceedings of the European Conference on Computer Vision (ECCV) Workshops (pp. 0-0).s

[4] <https://sites.google.com/view/visionlearning/databases/parking-lot-database>

[5] Kim, J. A., Sung, J. Y., & Park, S. H. (2020, November). Comparison of Faster-RCNN, YOLO, and SSD for real-time vehicle type recognition. In 2020 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia) (pp. 1-4). IEEE.

- [6]** Snegireva, D., & Perkova, A. (2021, September). Traffic Sign Recognition Application Using Yolov5 Architecture. In 2021 International Russian Automation Conference (RusAutoCon) (pp. 1002-1007). IEEE.
- [7]** Snegireva, D., & Kataev, G. (2021, September). Vehicle Classification Application on Video Using Yolov5 Architecture. In 2021 International Russian Automation Conference (RusAutoCon) (pp. 1008-1013). IEEE.
- [8]** Cygert, S., & Czyżewski, A. (2020). Toward robust pedestrian detection with data augmentation. IEEE Access, 8, 136674-136683.